

RedEye Harness — Improvements & Capabilities

A detailed, implementation-grounded write-up of what *redeye* (the AI-RedEye harness) is, and the concrete improvements it makes over a naive "ask-an-LLM-to-find-bugs" SAST loop. The emphasis is on the three things that make or break AI-driven SAST in production:

1. **Fewer false positives**
2. **Less hallucination** (no invented files, lines, sinks, or CWEs)
3. **Reduced cost** (token spend bounded and pruned cheaply)

...plus the operational, auditability, and portability improvements that make it usable as both a researcher's deep-dive and a PR gate.

RedEye is a derivative of Visa's **Vulnerability Agentic Harness (VVAH)** (Apache-2.0), substantially reworked and extended. See `NOTICE` and the `README` attribution.

1. The problem RedEye is built to solve

LLM-driven SAST is *noisy by default*. A frontier model asked "find vulns in this repo" will happily:

- cite files and line numbers that do not exist,
- hallucinate sinks/sources the code never calls,
- assert a CWE the code does not exhibit,
- and write confident "remediations" for bugs that were never there.

RedEye's core design principle: **put cheap, deterministic checks in front of every finding** so that what reaches a human cites real code and has survived every inexpensive check the harness can run. The model is allowed to be *aggressive about discovery* precisely because deterministic layers carry the precision burden.

2. Architecture at a glance

A 14-stage pipeline grouped into three phases. Critically, **five stages are fully deterministic and never call an LLM** (s1b structural pre-index, s4b grounding pass, s5 policy gate, s7 dedupe, and the voting *tally* in s6), plus the deterministic s8c outcome verifier.

Phase	Stages	Purpose
A — Discovery & Modeling	S1, S1b (DET) , S2, S3	Attack-surface mapping, structural pre-index, STRIDE threat model, hunting plan

B — Deep Dive & Verification	S4, S4b (DET) , S5 (DET) , S6 (+vote tally DET), S6.5	Multi-lens research, grounding, policy gate, adversarial review + voting, validator
C — Synthesis, Chaining & Reporting	S7 (DET) , S8, S8b, S8c (DET) , S9	Dedupe, exploit chaining, PoC gate, outcome verification, SARIF/MD emit

Reference: docs/architecture.md, docs/REDUCING_HALLUCINATIONS.md.

3. Improvement: fewer false positives

False positives are attacked with a *defense-in-depth* stack, ordered so the cheap deterministic filters run first and the model never wastes tokens on a finding that already failed a free check.

Multi-agent voting (S6) — the biggest LLM-side lever. Voting runs *after* the adversarial reviewer rewrites the attack chain, so voters score the refined finding, not raw lens output. A finding survives only if it clears a configurable quorum (default ≥ 2 confirms). N-of-M voting specifically kills *correlated* false positives — but only when voters disagree, which is why the docs recommend **cross-vendor voters** (e.g. Anthropic + OpenAI + a local model). Identical models make identical mistakes.

Single-pass validator (S6.5). A cheap, distinct TP/FP gate (Haiku / Gemini Flash class). It receives a compact dossier — deterministic grounding evidence + the taint shape — and returns `confirm` / `reject` / `uncertain`. The dossier shape is deliberate: it prevents the validator from rationalising away a hallucination.

Policy gate (S5, deterministic). Drops findings in test/vendor paths, findings with no remediation, and findings below the severity floor — zero LLM cost.

Deduplication (S7, deterministic). Merges the same root cause reported by multiple lenses, so one bug isn't counted (and reported) five times.

Outcome verification (S8c, deterministic). A final **K-of-N verdict over five independent upstream signals**: grounding, taint completeness, concrete PoC, reachability, and voter/validator agreement. Because it is *temperature-free*, it suppresses false positives even on models that reject a `temperature` knob (e.g. Opus / the `claude` CLI) where voting degenerates to a no-op. Unverified findings are flagged, or dropped when its `strict` parameter is set.

Baseline acceptance (.redeye-baseline.yaml). Operators accept reviewed findings once; their stable fingerprint (`sha256(cwe + path + start_line + skill)`) keeps them filtered out of every future report. The fingerprint is robust to LLM rewording but pinned to the structural identity of the bug. See `redeye/baseline.py`.

Feedback loop. Reviewer TP/FP marks from PR comments persist to a local SQLite store (`~/redeye/scans.db`) and are injected into the next scan's lens prompts as in-context calibration — the harness learns away from the FPs it produced last time.

4. Improvement: less hallucination (7-layer control)

This is the headline of the 0.3 redesign. Seven layers sit in front of the report; a finding cannot reach the operator without surviving every cheap deterministic check. (docs/REDUCING_HALLUCINATIONS.md.)

Layer	Stage	What it kills	LLM cost
Structural pre-index	S1b	Invented routes/sources/sinks/secrets	Zero (regex/AST)
Taint-flow schema	S4 contract	Hand-wavy findings with no source→sink theory	Zero (parse-time)
Grounding pass	S4b	Cited file/line that doesn't exist; no CWE-family tokens nearby	Zero (file I/O)
Validator auto-reject	S6.5	Findings already tagged hallucinated by S4b	Zero (skips the call)
PoC gate	S8b	Findings the model can't write a concrete payload for	Cheap LLM
Multi-agent voting	S6	Findings only one voter agrees with	LLM × N
Outcome verifier	S8c	Findings failing the K-of-N deterministic verdict	Zero

Layer 1 — Structural pre-index (S1b). Before *any* LLM call, regex + AST extract a deterministic inventory of the target's real **routes** (FastAPI/Flask/Express/Spring/Django), **sources** (request body, query, env, stdin, argv, Kafka), **sinks** (SQL execute, subprocess, eval/exec, unsafe deserialization, `os.system`, weak crypto, JWT misverification, SSRF, path traversal), and **secrets** (cloud/API tokens, PEM keys, high-entropy assignments). This inventory becomes ground truth: lenses reason "given these *real* sinks, which combos are dangerous?" instead of "imagine where bugs might be." A whole class of hallucination disappears because the model can no longer invent code that isn't there. (redeye/structural.py.)

Layer 2 — Explicit taint shape. Every lens response must carry a taint block: `source`, `source_location`, `sink`, `sink_location`, `sanitizer_missing`, `sanitizers_observed`, and an ordered `taint_path` of file:line steps. A response without `source` + `sink` is rejected at parse time. This forces the model to hold a *theory* of the bug, and `sanitizers_observed` lets it disprove its own suspicion (e.g. "I saw an authz check that defeats this IDOR — not emitting").

Layer 3 — Grounding pass (S4b, deterministic). For each finding it (1) resolves the cited path (rejects `..` escapes / nonexistent files), (2) checks the line is within the file, and (3) reads a ±5-line window for CWE-family tokens (CWE-89 → `execute/query/cursor`; CWE-78 → `subprocess/shell/os.system`). Outcomes:

Outcome	Tag	Default	<code>findings</code>
All pass	<code>grounded: true</code>	keep	keep
File/line missing	<code>hallucinated:bad-path / bad-line</code>	keep + tag (validator auto-rejects)	drop

Tokens missing	weak-evidence	keep, cap confidence ≤ 0.5	keep, cap ≤ 0.5
----------------	---------------	---------------------------------	----------------------

It also emits an Evidence row per finding (which checks passed/failed), surfaced in the Markdown report so reviewers can audit the harness's reasoning.

Layer 4 — Validator auto-reject. S6.5 reads `finding.tags` *before* spending a token. Anything tagged `hallucinated:bad-path/bad-line` is auto-rejected — no point asking a model to confirm a bug citing fictional code.

Layer 5 — PoC gate (S8b). The model must write a *concrete* exploit string. Syntactic checks require at least one of {quoted payload, URL, HTTP verb, injection metachar, `../`, `$`{, `<script`, a SQL keyword}. Placeholders (`<exploit_here>`, `malicious_input`) are rejected. No concrete PoC → demote one severity notch (or drop with `--require-poc`). Rationale: a real bug usually has an obvious payload; if the model can't write one, it probably doesn't understand the bug.

Layers 6–7 — Voting + outcome verification (see §3).

Per-finding evidence trail + hallucination metrics. Every run emits counters (in `run_manifest.json` and the report header):

```
{
  "raw_lens": 24,
  "ungrounded_dropped": 6,
  "ungrounded_downgraded": 4,
  "validator_rejected": 3,
  "voted_out": 2,
  "missing_poc": 1,
  "outcome_unverified": 0
}
```

These make the noise the harness pruned *visible* — a high `ungrounded_dropped` means a lens is hallucinating paths (tighten the prompt / use a stronger model); a high `voted_out` means voting is earning its keep. The metrics also feed the feedback loop.

5. Improvement: reduced cost

Token spend is the dominant cost in agentic SAST. RedEye bounds and prunes it at multiple points:

Cheap-first filter ordering. The pipeline deliberately runs **zero-LLM deterministic layers first** (S1b, S4b, S5, S7, S8c). A finding that cites a nonexistent path is dropped by file I/O *before* any model is asked to adversarially review, validate, or write a PoC for it. This is the single biggest structural cost saving.

Per-stage budget caps. Every stage carries `max_budget_usd` in its profile; the sum is the worst-case spend per scan. When a stage runs out of budget it stops calling the backend but keeps its partial result. For most teams `s4_research` and `s6_adversarial` dominate the bill, so those caps are the main knobs. (`docs/configuration.md`.)

Cost/scope estimate before spending. `redeye estimate --repo PATH` produces a **no-spend dry run** — file counts, language mix, and a rough USD budget per stage — so operators (and AI agents, per `AGENTS.md`) preview cost and confirm before running `scan`.

Scope control / DoS limits. Scope (redeye/scope.py) bounds what the pipeline reads: `--diff-only/--pr-base` scopes a PR scan to only changed files via `git diff`; `--max-files`, `--max-file-bytes`, `--max-total-bytes` cap volume; `--exclude-path` drops test/vendor noise; and default ignore dirs (`node_modules`, `vendor`, `dist`, `.venv`, ...) plus an interesting-extensions allowlist mean only relevant source is ever sent to a model.

Incremental structural cache. The deterministic pre-index is cached per file by (path, size, mtime_ns) in `~/.redeye/cache/structural/<target>.json`. On the next scan, unchanged files are served from cache and only changed files are re-scanned. Cache hits never go stale silently (a 1-byte/1-ns change rebuilds). (redeye/cache.py.)

Cheap models where cheap is enough. The validator (S6.5) and PoC gate (S8b) are explicitly Haiku- / Gemini-Flash- class jobs, reserving expensive models for the research lenses and adversarial review. Roles map to backends per-stage in the profile, so you pay premium rates only where they matter.

Validator auto-reject (also a cost win). Skipping the LLM call for already-hallucinated findings (\$4, layer 4) saves tokens on exactly the findings least worth spending them on.

Zero-cost mock backend. `--preset quick / make demo` runs the full 14-stage pipeline deterministically with **no API keys and no network calls** — ideal for CI smoke tests and demos at literally zero LLM cost.

6. Other notable improvements

Dual-mode, preset-driven UX. One tool is both a researcher's deep-dive and a PR gate. Presets are default-overlays; any explicit flag still wins.

Preset	Backend	Scope	Grounding	Use when
quick	mock (no cost)	small	lenient	trying the tool, CI smoke tests
pr	active profile	files changed vs base	strict	every PR
ci	active profile	whole repo, DoS-capped	strict	nightly cron
deep	active profile	unlimited	lenient	research deep-dive

Multi-cloud / offline backends. Anthropic (CLI + SDK), OpenAI / OpenAI-compatible, **AWS Bedrock**, **Google Vertex (Gemini)**, **Ollama (local)**, and the deterministic **mock** — auto- detected, with backends falling back to mock on error so a run still completes. No single provider is a hard dependency.

Native CI/CD + GitHub integration. Drop-in GitHub Actions workflow runs PR scan + full scan + feedback collection. Findings emit **SARIF 2.1.0** with `security-severity` and `taint codeFlows`, so GitHub Code Scanning renders inline taint traces; PR comments carry `[ ] True Positive / [ ] False Positive` checkboxes that drive the feedback loop. Webhooks (Slack/Teams/Discord/generic) support optional HMAC signing.

Auditability by default. Every run writes `run_manifest.json`: tool version, profile, config hash, target SHA, per-stage cost, and quality metrics — an immutable audit trail.

CVSS support. Findings can carry `cvss_vector` + `cvss_score`; both flow into SARIF along with `security-severity`.

Extensibility. New research lenses, backends, structural patterns, CWE token catalogs, and output emitters are all documented extension points (`docs/architecture.md` §Extending). Lens prompts read the structural inventory verbatim, so a new regex pattern is immediately visible to the model.

Safety posture. `AGENTS.md` constrains AI agents driving the tool: don't edit the harness to make a scan pass, don't invent credentials, don't scan unnamed repos, and never treat findings as ground truth. `REDEYE_NO_NETWORK=1` hard-refuses network-using backends.

7. Improvements vs. a naive LLM-SAST loop (summary)

Dimension	Naive "ask the LLM"	RedEye
Invented file/line	Common	Dropped/tagged deterministically at S4b
Invented sinks/CWE	Common	Constrained by S1b structural ground truth
False positives	High	Voting + validator + policy gate + S8c K-of-N
Cost control	Unbounded	Per-stage budgets, diff scope, cache, cheap-first ordering, mock
Evidence	None	Per-finding PASS/FAIL evidence trail
Repeat FPs	Re-reported every run	Baseline + SQLite feedback loop suppress them
Auditability	None	<code>run_manifest.json</code> (version, hash, SHA, costs, metrics)
Output	Free text	SARIF 2.1.0 + Markdown + manifest, GitHub-native

8. Limitations (honest framing)

- **LLM-generated, non-deterministic.** Findings are *triage candidates*, not confirmed vulns; two runs can differ unless every backend is mock.
 - **Token-hungry.** Caps are per-stage — use `estimate` and the DoS limits.
 - **Elevated privilege.** Run only against trusted repos by authorized operators.
 - **Feedback loop is local-first.** SQLite at `~/redeye/scans.db`; a Databricks backend is on the roadmap.
-

9. References (in-repo)

- `README.md` — overview, quickstart, presets, capabilities
- `docs/architecture.md` — stage-by-stage pipeline + contracts
- `docs/REDUCING_HALLUCINATIONS.md` — long-form rationale for each filter
- `docs/configuration.md` — profiles, budgets, voter selection
- `docs/CI_AND_FEEDBACK.md` — CI/CD wiring + feedback loop
- `CHANGELOG.md` — 0.3.0 release notes
- Source: `redeye/structural.py`, `redeye/grounding.py`, `redeye/scope.py`, `redeye/cache.py`, `redeye/baseline.py`, `redeye/schema.py`